Continuous Integration Report

ENG1 Cohort 1 Team 3

Almira Demir
Mansi Deshkar
Laina Ha
James McDermott
Harry McLean
Vadim Mitko

Summary on Continuous Integration (CI) in UniSim

In our project of UniSim, we use Continuous Integration (CI) to help to synchronize the coding progress across team member's machines and help to detect any problems early on and prevent bugs from making it to production. This is necessary to reduce the cost and impact of fixing bugs, as it is easier to fix a bug during development and testing rather than during production.

After a discussion with all the team members, we have decided that the following CI integration methods must be implemented:

- UniSim's executables must be available from the Github repository at all times. If a new commit is made, a new executable must be created and be easily accessible.

- If a new commit is made, the game must be automatically built and tested. Whether the new version passed or failed the testing must be clear; The testing report must be accessible by all members of the team.

- Game releases must be automatic and happen only if the game is tested and deemed to be without flaws.

We decided that the automation of testing, building and releasing a game is an appropriate CI method as it stems from the DRY principle of "do not repeat yourself". If we decide to automate testing, building and releasing, then each individual member of the team will spend less time compiling, testing, and manually writing a sufficient release description each time they create a new release.

The CI automation and wide availability of test results and different versions of executables allows for better and quicker collaboration within the team, truly showing how efficient Agile Software Development can be if done together with CI.

Having these resources available allows our team to efficiently cross validate and check commits, and, in a case where commits fail the tests, to notify the person responsible to check the code and perhaps rewrite it / fix it.

Moreover, having executables right in the Github repository makes a potential user's evaluation easier and more intuitive, as they may access the game from their own machine by only downloading a single already-compiled file (.jar file).

In our GitHub repository, we have set up an automatic CI workflow system, which uses a yaml configuration file to describe which Github actions must be performed on a push to the repository. Currently, our workflow file does the following tasks:

1. Using latest version of Ubuntu, build the game on a VM
2. Upload the Jacoco testing report to the Github repository
3. Upload the game's executable (JAR) to the Github repository
4. If during commit a release version tag is specified, the Github repository will automatically issue a new version of the game

Github actions are executed automatically every time the code is committed and pushed to the repository. Currently, when a change is made, committed and pushed, the new version of the game will be built on a Linux virtual machine together with the game's tests. If the new commit breaks the game mechanics, the tests wouldn't pass and the game wouldn't build. That becomes visible almost immediately after a commit is pushed and necessary action can be taken quickly.

Together with the build, our workflow publishes two more "build artifacts": Jacoco test report and JAR executable file.

Jacoco test report is essentially a zip folder with index.html viewable in any browser which has extensive description of which tests did the new commit fail and which tests did it pass. This allows any other member on the team to view the test report immediately after a new commit has been pushed. The tests used in our Jacoco test report are the ones written in "headless" build, which is built simultaneously with root build.

With every commit, our Github workflow also produces a JAR executable file, ready to be downloaded and launched. Having a JAR executable file alongside with every release helps to investigate some of the less obvious bugs and problems that might be in our game, such as unclear / off-centered labels of the UI interface. Moreover, since we are using the Agile Software Development model, having a new JAR file is crucial because this way the new prototype can be assessed and modified very quickly.

Moreover, apart from these two build artifacts, our yaml configuration file also specifies that if someone adds a version tag to a commit, that will trigger a new game release with a newly ready JAR executable for it. This automatizes the version control in our Github repository and allows our team members to spend more time on delivering high quality code.